



End-to-End AI System Sprint

Technical Assessment — AI Engineer Role

DOTMappers IT Pvt. Ltd.

Duration	Format	Submission	Walkthrough
48 Hours	GitHub Repository	GitHub link + README	30 min post-submission

1. Overview

This assessment evaluates your ability to independently architect, build, and deliver a working AI system — from a problem statement to a functional prototype — within a 48-hour window. This mirrors what is expected of the AI Engineer role at DOTMappers: translating a business problem into a production-grade AI solution with minimal handholding.

2. Problem Statement

You are given a customer support ticket dataset (CSV). Build an AI-powered system that does all the following:

- Ingest the CSV data and make it queryable.
- Answer natural language questions about the data (e.g., "How many critical tickets are unresolved?", "Which agent has the lowest average customer rating?").
- Detect and flag anomalies (e.g., tickets with abnormally long resolution times, unresolved high-priority tickets older than 24 hours).
- Expose the functionality via a REST API **AND** a minimal UI — **Both are required.**

The system must use an LLM for natural language understanding. You may use any free-tier or locally run model (Ollama, Groq free tier, HuggingFace Inference API, etc.).

3. Dataset

3.1 File Provided

Filename: support_tickets.csv | Total Rows: 500 | Format: UTF-8 CSV

3.2 Schema Preview

ticket_id	created_at	category	priority	status	resp_time_hrs	resol_time_hrs	agent_id	cust_rating	issue_summary
TKT-001	2024-01-03 09:12	Billing	High	Resolved	0.5	2.3	AGT-04	4	Incorrect charge on invoice
TKT-002	2024-01-03 11:45	Technical	Critical	Escalated	1.2	18.5	AGT-07	2	Login failure after update
TKT-003	2024-01-04 08:30	General	Low	Resolved	3.1	5.0	AGT-02	5	Request for product docs
TKT-004	2024-01-04 14:22	Technical	High	Resolved	0.8	4.7	AGT-04	3	API timeout in production
TKT-005	2024-01-05 10:05	Billing	Medium	Open	2.0		AGT-09		Refund not processed
... (500 rows total)									

3.3 Column Descriptions

Column	Type	Description
ticket_id	String	Unique ticket identifier
created_at	Datetime (YYYY-MM-DD HH:MM)	Ticket creation timestamp
category	String (Billing / Technical / General)	Issue category
priority	String (Low / Medium / High / Critical)	Ticket urgency level
status	String (Open / Resolved / Escalated)	Current ticket status
response_time_hrs	Float	Hours from creation to first agent response
resolution_time_hrs	Float (null if unresolved)	Hours from creation to resolution
agent_id	String	Assigned support agent identifier
customer_rating	Integer 1–5 (null if unresolved)	Post-resolution satisfaction rating
issue_summary	String (free text)	Brief description of the issue reported

4. Deliverables

Your GitHub repository must include all the following:

- **1.** Working system fulfilling all four requirements in Section 2.
- **2.** REST API (FastAPI or equivalent) with at least 3 endpoints — NL query, anomaly detection, health check — OR a minimal UI (Streamlit, Gradio, or similar) covering the same functionality.
- **3.** README.md covering: setup instructions, architecture overview, model/tools used, example queries with outputs, known limitations.
- **4.** requirements.txt or equivalent so the evaluator can run the system locally.

5. Technical Constraints

- Language: Python only.
- LLM: Must use an LLM for NL query handling. Allowed: Ollama (local), Groq free tier, HuggingFace Inference API free tier, or any locally runnable model.
- No paid APIs or services. The evaluator must be able to run your system at zero cost.
- The system must start with a single command (e.g., docker-compose up or uvicorn main:app).

6. Evaluation Criteria

Criterion	What We Look For	Weight
Functionality	All 4 requirements work as described	30%
Architecture & Design	Component choices are reasoned, not accidental	25%
Code Quality	Clean, modular, readable, with error handling	20%
LLM Integration Quality	Prompt design, output structuring, edge case handling	15%
README & Documentation	Clear setup, architecture explanation, example outputs	10%

7. Timeline

Phase	Time Window	Focus
Planning	Hours 0–4	Read brief, design architecture, choose tools and LLM
Core Pipeline	Hours 4–16	Data ingestion, LLM integration, NL query handling, anomaly logic

Phase	Time Window	Focus
API / UI Layer	Hours 16–32	Expose endpoints or UI, input validation, error handling
Polish & Submit	Hours 32–48	Testing, README, code cleanup, GitHub submission

8. Submission Instructions

- Share your GitHub repository link via email to: RajathKumar@dotmappers.in
- Subject line: [AI Engineer Assessment] — Your Name
- Deadline: 48 hours from the time this document is received.
- A 30-minute architecture walkthrough call will be scheduled after submission.

9. Sample Queries

The following are indicative queries your system should handle. The evaluator will use their own queries during the walkthrough.

- "How many tickets are currently open?"
- "Which agent resolved the most tickets this month?"
- "Show me all Critical tickets not resolved within 12 hours."
- "What is the average customer rating for Technical category tickets?"
- "Are there any anomalies in resolution times this week?"

10. Notes

This assessment is intentionally open-ended. There is no single correct architecture. You will be evaluated on the reasoning behind your choices as much as the working code. The 30-minute post-submission walkthrough is your opportunity to explain trade-offs, what you would improve with more time, and how you would scale the system.

Good luck.